



第13章 Java Web开发常用功能

- 文件上传
- 分页处理
- JavaMail
- 树状菜单



学习目标

- 理解Java Web开发常用的功能，如文件上传、分页、E-mail、树形菜单的基本概念、功能和优点。
- 掌握文件上传、分页、E-mail、树形菜单的实现以及在实际开发中的应用

第13章 Java Web开发常用功能

- 文件上传
- 分页处理
- JavaMail
- 树状菜单

文件上传

多数文件上传都是通过表单形式提交给后台服务器的，因此，要实现文件上传功能，就需要提供一个文件上传的表单，而该表单必须满足以下3个条件：



- form表单的method属性设置为post;
- form表单的enctype属性设置为multipart/form-data;
- 提供

当form表单的enctype属性为multipart/form-data时，浏览器就会采用二进制流来处理表单数据，服务器端就会对文件上传的请求进行解析处理。

文件上传

文件上传表单示例如下

```
<form action="uploadUrl" method="post" enctype="multipart/form-data">  
  <input type="file" name="filename" multiple="multiple" />  
  <input type="submit" value="文件上传" />  
</form>
```

multiple属性是
HTML5中新属性，
可实现多文件上传

文件上传

- Servlet3.0规范之前，在Java的Web开发中没有对文件的上传进行封装，需要自己开发一个Servlet或者JavaBean处理上传或下载的任务。
 - 从HttpServletRequest中获得客户端请求的输入流。
 - 通过输入流读取指定的文件，将文件保存在指定的位置。



文件上传

- **jspSmartUpload 组件**是一个免费使用的文件上传组件，它的使用简单，功能齐备。
- 通过该组件，可以获得上传文件的全部信息(包括文件名、大小、类型、扩展名、文件数据等)，同时还可以对上传文件的大小、类型等方面进行限制。
- 在使用时需要把该组件的jar文件放到站点WEB-INF目录的lib中。



文件上传

- jspSmartUpload组件中包括File、Files、Request、SmartUpload等类。
 - File: 包括上传文件的所有信息, 如: 上传的文件名、大小、扩展名、文件数据等。
 - Files: 所有上传文件的集合。从中可以获得上传文件的数目、大小等。
 - Request: 相当于JSP中的request对象, 如果在上传的表单中还有其它的表单项的值, 必须通过jspSmartUpload组件中的Request对象获取。
 - SmartUpload: 完成文件上传。 。



单元项目1-采用jspSmartUpload 组件上传文件

- 项目构思：采用jspSmartUpload 组件将客户端的doc和txt文件上传到Web服务器。
- 项目设计：
 - 创建一个JSP文件用于客户端的文件上传界面。
 - 将jspSmartUpload组件加入到项目中。
 - 创建一个Servlet用于处理上传的文件。



单元项目1-采用jspSmartUpload 组件上传文件

- 项目实施：

- 创建一个JSP文件upload.jsp：（主要代码如下）

```
<form action="upload" method="post"
      enctype="multipart/form-data">
```

```
文件描述：<input type="text" name="desc"
                  size="20" maxlength="80">
```

```
文件名称：<input type="file" name="file"
                  size="20" maxlength="80">
            <input type="submit" value="上传">
```

```
</form>
```

注意：form表单中必须添加**method="post"**与**enctype="multipart/form-data"**属性。



单元项目1-采用jspSmartUpload 组件上传文件

- 项目实施：
 - 将jsmartcom_zh_CN.jar文件拷贝到项目目录的WEB-INF/lib下。
 - 在Servlet中处理上传文件。



单元项目1-采用jspSmartUpload 组件上传文件

- 项目实施：

- 在Servlet中处理上传的文件主要代码：

```
SmartUpload mySmartUpload = new SmartUpload();
int count = 0;//上传文件的数量
try {
    mySmartUpload.initialize(this.getServletConfig(), request, response);
    // 限制每个上传文件的最大长度。
    mySmartUpload.setMaxFileSize(50 * 1024 * 1024);
    // 设定允许上传的文件（通过扩展名限制），仅允许doc,txt文件。
    mySmartUpload.setAllowedFilesList("doc,txt");
    mySmartUpload.upload();
    //获得上传的文件
    File myfile = mySmartUpload.GetFiles().getFile(0);
    //获得上传文件的名字
    String fileName = myfile.getFileName();
    //保存文件的目录
    count = mySmartUpload.save("/upload");
    //获得文件的描述信息
    Request re=mySmartUpload.getRequest();
    String desc=re.getParameter("desc");
    out.println(count + " file uploaded.<br>");
    out.println("file description:"+desc);
} catch (Exception e) {
    out.println("Unable to upload the file.<br>");
    out.println("Error : " + e.toString());
}
```

文件上传

- jspSmartUpload组件使用灵活，适合较小文件的传输，如果传输数据较大，则采用 **commons-fileupload** 组件。
- commons-fileupload 组件的下载地址为 http://commons.apache.org/proper/commons-fileupload/download_fileupload.cgi。
- 使用 commons-fileupload 组件的时候需要将其 jar 文件放到站点目录 WEB-INF/lib 下同时还有加入 commons-io 的 jar 包。其下载的地址是 http://commons.apache.org/proper/commons-io/download_io.cgi。

文件上传

- org.apache.commons.fileupload包中包括了在commons-fileupload组件所有的类。
 - DiskFileItemFactory: 代表本地的磁盘文件。
 - FileItem: 代表每组数据的接口。
 - ServletFileUpload: 获得上传文件。



单元项目2-采用commons-fileupload组件上传文件

- 项目构思：采用commons-fileupload组件将客户端的doc和txt文件上传到Web服务器。
- 项目设计：
 - 创建一个JSP文件用于客户端的文件上传界面。
 - 将commons-fileupload组件加入到项目中。
 - 创建一个Servlet用于处理上传的文件。

单元项目2-采用commons-fileupload组件上传文件

- 项目实施：
 - 创建一个JSP文件fileupload.jsp。
 - 将commons-fileupload-1.3.1.jar和commons-io-2.4.jar文件拷贝到项目目录的WEB-INF/lib下。



单元项目2-采用commons-fileupload组件上传文件

- 项目实施：
 - 创建一个Servlet实现文件上传。

```
//实例化一个硬盘文件工厂,用来配置上传组件ServletFileUpload
DiskFileItemFactory factory = new DiskFileItemFactory();
factory.setSizeThreshold(4096); // 设置缓冲区大小,这里是4kb
ServletFileUpload upload = new ServletFileUpload(factory); //用以上工厂实例化上传组件
upload.setSizeMax(4194304); // 设置最大文件尺寸,这里是4MB
String uploadPath = this.getServletContext().getRealPath("/upload");// 设置上传的地址
List items = upload.parseRequest(request); // 得到所有的上传文件
Iterator it = items.iterator();
while (it.hasNext()) { //逐条处理
    FileItem fi = (FileItem) it.next(); //得到当前文件
    //检查当前项目是普通表单项目还是上传文件
    if (fi.isFormField()) { //如果是普通表单项目,显示表单内容。
        if ("desc".equals(fi.getFieldName())) {
            out.println("file description:"+fi.getString());
        }
    } else { //获得上传的文件
        String path = fi.getName(); // 得到文件的完整路径
        String filename = path.substring(path.lastIndexOf("\\") + 1); // 得到去除路径的文件名
        fi.write(new File(uploadPath, filename)); //将文件保存在Web目录的upload文件夹中
        out.println(filename + " file uploaded.<br>");
    }
}
```



单元项目3-采用Servlet3.0上传文件

- Servlet3.0较之以往版本，增加了一些实用的新功能，其中一个比较大的改进就是HttpServletRequest对象增加了对文件上传的支持。
- HttpServletRequest提供了如下两个方法来处理文件上传：
 - (1) Part getPart(String name)：根据名称来获取文件上传域。
 - (2) Collection<Part> getParts()：获取所有的文件上传域。
- 这两个方法都涉及到了一个新的API——Part，每一个Part对象对应于一个文件上传域，它是javax.servlet.http.Part类型的对象，该对象提供的主要方法如下：
 - (1) String getContentType()：获得上传文件的内容类型。
 - (2) long getSize()：获得上传文件的大小；
 - (3) String getHeader(String name)：获取指定name的头部信息，例如：头部“content-disposition”的信息中包含上传文件的文件名。
 - (4) InputStream getInputStream()：获得读取上传文件内容的输入流。
 - (5) void write(String filename)：将上传文件写到磁盘。



第13章 Java Web开发常用功能

- 文件上传
- 分页处理
- JavaMail
- 树状菜单



分页处理

- 在Web开发中，分页处理显示数据是最基本的功能。分页显示是将数据库中的数据依次部分地显示出来。通过分页处理可以提高页面访问速度，美化页面。在实际开发中有很多**分页的解决方案**，大致可以分为以下两种：
 - 利用结果集 (ResultSet) 来处理。
 - 采用SQL语句处理。



分页处理

- 利用结果集 (ResultSet) 来处理。
 - 通过ResultSet的absolute()方法获得指定行位置的记录。
 - 当用户第一次请求数据查询时，就执行SQL语句查询，获得的ResultSet对象及其要使用的连接对象都保存到其对应的会话对象中。
 - 以后的分页查询都通过第一次执行SQL获得的ResultSet对象定位取得指定行位置的记录。
 - 最后在用户不再进行分页查询时或会话关闭时，释放数据库连接和ResultSet对象等数据库访问资源。
- 这种方式对数据库的访问资源占用比较大，并且其利用率不是很高。
- 优点是减少了数据库连接对象的多次分配获取，减少了对数据库的SQL查询执行。



分页处理

- 采用SQL语句处理。
 - 在用户的分页查询请求中，每次可取得查询请求的行范围的参数。
 - 使用这些参数取得指定行范围的SQL查询语句，并执行SQL查询。
 - 把查询的结果返回给用户，最后释放数据库访问资源。
- 采用这种方式需要每次请求时都要执行数据库的SQL查询语句；对数据库的访问资源使用完毕就立即释放，不占用数据库访问资源。
- 其缺点是：
 - 对不同的数据库使用的查询语句是不同的。
 - 每次均执行数据库SQL查询操作。对数据库有一定的影响。



单元项目3-分页功能的实现

- 项目构思：采用MVC模式实现分页功能。将表中的数据按照分页的方式显示到页面。
- 项目设计：根据MVC的分层原则，将分页显示的功能分为3个层次。
 - 模型层：用于代表数据表的数据Bean、用于在页面传输数据的PageBean、用于获得分页数据的逻辑Bean。
 - 控制层：用Servlet完成分页控制操作。
 - 视图层：采用JSP页面显示分页数据。



单元项目3-分页功能的实现

- 项目实施：

- 创建用于在页面中传输数据的PageBean。

```
public class PageBean {  
    private int curPage;           //当前页数  
    private int totalPages;        //总页数  
    private int totalRows;         //总行数  
    private int pageSize;          //每页显示行数  
    private List data;             //每页显示的数据  
    .....省略get、set方法  
}
```

- PageBean的主要功能是在视图和控制层之间传输数据，里面封装了需要显示的分页信息和分页的数据。



单元项目3-分页功能的实现

- 项目实施：
 - 创建数据Bean: User Info。
 - 创建连接工具类: DBUtil 。



单元项目3-分页功能的实现

- 项目实施：

- 创建逻辑Bean User Info，获得分页数据。

//返回数据库中分页用户信息的方法

```
public PageBean getUserList(int curPage) {  
    String sql = "select * from users";  
    PageBean pb = db.getPageBean(sql, null,  
curPage);  
    return pb;  
}
```



单元项目3-分页功能的实现

- 项目实施：

- 创建Servlet控制层，完成分页的控制操作。

// 获得要显示的页数

```
String page = request.getParameter("page");
```

// 当前的页数

```
int curPage = 1;
```

// 如没有传入的页数

```
if (page != null && page.length() > 0) {
```

```
    curPage = Integer.parseInt(page);
```

```
}
```

// 调用模型

```
UserInfo ui = new UserInfo();
```

// 将PageBean放入到request中转发

```
request.setAttribute("pageBean", ui.getUserList(curPage));
```

```
request.getRequestDispatcher("userInfoListByPage.jsp").forward(request,  
response);
```

单元项目3-分页功能的实现

- 项目实施：

- 创建视图层

```
<table border="1" align="center" width="50%">
```

```
<tr><th>序号<th>用户名<th>昵称</tr>
```

```
<c:forEach var="user" items="{pageBean.data}" varStatus="vs">
```

```
<tr>
```

```
    <td align="center"><c:out value="{vs.count}" /></td>
```

```
    <td align="center"><c:out value="{user.username}" /></td>
```

```
    <td align="center"><c:out value="{user.nickname}" /></td>
```

```
</tr>
```

```
</c:forEach>
```

```
</table>
```



第13章 Java Web开发常用功能

- 文件上传
- 分页处理
- **JavaMail**
- 树状菜单

JavaMail

- Email是通过计算机网络交换的电子媒体信件。
- 接收电子邮件：申请ISP邮件服务器的一个电子信箱，由ISP邮件服务器负责电子邮件的接收。一旦有用户的电子邮件到来，ISP邮件服务器就将邮件移到用户的电子信箱内，并通知用户有新邮件。
- 发送电子邮件：电子邮件首先从用户计算机发送到ISP邮件服务器，再到Internet，再到收件人的ISP邮件服务器，最后到收件人的个人计算机。
- 电子邮件在发送与接收过程中都要遵循SMTP、POP3等协议，这些协议确保了电子邮件在各种不同系统之间的传输。其中，SMTP负责电子邮件的发送，而POP3则用于电子邮件的接收。

JavaMail

- Email的相关协议

- **简单传输协议 (SMTP)**: Simple Mail Transfer Protocol, 是电子邮件从客户机传输到服务器或从某一个服务器传输到另一个服务器使用的传输协议, 通信基于TCP协议端口25。
- **邮箱协议 (POP3)**: 是将个人计算机连接到Internet的邮件服务器和下载电子邮件的协议。它允许用户从服务器上把邮件存储到本地主机 (即自己的计算机), 同时删除保存在邮件服务器上的邮件, 这个协议工作的TCP协议端口是110。
- **网络消息访问协议 (IMAP)**: 与POP3协议类似, 也是提供面向用户的邮件收取服务。它使用TCP端口是143。这两个协议的不同是IMAP将很多客户端的功能转移到服务器端。
- **因特网邮件扩展标准 MIME**: 本身不是邮件传输协议, 但是对于传输内容的消息, 附件以及其它内容定义了格式。例如: 扩充了非文本邮件主体、邮件主体不再局限于ASCII字符, 而是所有的字符等。



JavaMail

- JavaMail是JavaEE中的标准API，在它的接口中封装了与邮件服务器访问的详细过程。
- 采用JavaMail API可以在程序中以一种独立于平台和协议的方式发送和接受邮件。
- JavaMail还允许创建不同类型的邮件，比如普通的文本，或者带有附件的邮件，或者带有混合的二进制内容的邮件。
- JavaMail API主要包括四部分：Session、Message、Transport、和InternetAddress。

JavaMail

- Session

- Session类代表一个基本的邮件会话，其它的对象均依赖于Session。它是与一组用户邮件配置设置有关的方法提供者。通过配置这些属性完成客户机和服务器之间的交流。
- Session对象使用java.util.Properties获得配置的信息，通过getDefaultInstance()方法创建邮件会话。

例：

```
Properties props=new Properties();  
props.put("mail.host","邮件服务器的名字");  
.....  
Session session=Session. getDefaultInstance(props,null);
```



JavaMail

- Message

- Message类代表一个邮件消息，但这是一个抽象类，所以必须通过它的子类实现，通常采用 `java.mail.internet.MimeMessage`，它代表了一个标准的MIME风格的邮件消息，通过一个接受 `Session`对象的构造函数创建实例。

例: `MimeMessage message=new MimeMessage(session);`

- Message类的主要方法可以分为两部分。第一部分主要用于发送邮件，设置邮件的相关信息包括邮件的发送者、接受者、主题和发送时间等。第二部分用于接受邮件，用来获取邮件的相关信息。



JavaMail

- Transport

- Transport是消息发送传输类，提供了两个静态的send()方法发送消息，第一种使用一个Message对象和Address数组做参数，它发送消息给所有Address类定义的收件人。第二种形式只有一个Message对象参数，但是必须预先定义收件人的地址。

例：Transport.send(message);

- 还可以通过一个邮件会话获得Transport类的实例，通过调用connect()的方法，发送邮件。

例：

```
Transport tran=session.getTransport("smtp");
```

```
tran.connect("邮件服务器地址","用户名","密码");
```

```
tran.send(msg);
```



JavaMail

- InternetAddress

- InternetAddress代表用户的邮箱地址，它的父类是Address。可以通过传入一个正确的邮箱地址的构造函数创建实例。

例：

```
InternetAddress address=newInternetAddress("abc@sohu.com");
```

单元项目4-创建第一封电子邮件

- 项目构思：利用JavaMail API，通过程序将邮件发送到邮箱中。
- 项目设计：通过一个JSP页面将邮件发送到邮箱中。



单元项目4-创建第一封电子邮件

- 项目实施：
 - 将JavaMail API的jar包(javax.mail.jar)拷贝到站点的WEB-INF\lib文件夹中。
 - 在项目的JSP文件中，实现发送邮件的功能。



单元项目4-创建第一封电子邮件

- 项目实施：
 - JSP中的部分代码：

```
Properties props = new Properties();  
props.put("mail.host", "219.216.128.8"); //设置邮件服务器地址  
Session ses = Session.getDefaultInstance(props, null); //创建邮件会话对象  
//从Session创建MimeMessage  
MimeMessage msg = new MimeMessage(ses);  
//发送邮件的邮箱地址  
InternetAddress from=new InternetAddress("abc@neusoft.edu.cn");  
//接受邮件的邮箱地址  
InternetAddress to=new InternetAddress("a123@sohu.com");  
msg.setFrom(from); //设置发送邮件邮箱地址  
msg.addRecipient(Message.RecipientType.TO, to); //添加邮箱接受者  
msg.setSubject("first mail"); //设置邮件主题  
msg.setText("this is my first Mail"); //邮件内容  
Transport tran=ses.getTransport("smtp"); //获得传输类的实例  
tran.connect("219.216.128.8", "abc@neusoft.edu.cn", "password"); //建立连接  
tran.send(msg); //发送邮件  
out.println("邮件发送成功");
```



单元项目5-创建HTML格式的邮件

- 项目构思：
 - HTML格式的邮件可以给收件人更精彩的内容，HTML格式的邮件中可以插入视频、音频、图片、动画等多媒体的内容，HTML格式的邮件和文本类型的邮件发送的过程基本相同，只是在Message对象中传输的是HTML格式的文件，并且要设置邮件内容的格式。
 - 可以通过文件流读取要发送的HTML邮件，然后将内容添加到Message对象中，同时设置内容的格式。
- 项目设计：
 - 通过一个JSP文件将HTML格式的邮件发送到邮箱中。



单元项目5-创建HTML格式的邮件

- 项目实施：
 - 在发送Email的代码基础上添加代码：

```
//获得HTML文件路径
String source=application.getRealPath("/")+"\\\\"+"source.html";
String content="";
String line=null;
//创建文件输出流
BufferedReader br=new BufferedReader(new FileReader(source));
    while((line=br.readLine())!=null){
        content+=line;           //读取HTML文件内容
    }
br.close();
//把邮件内容添加到Message对象中，并且设置内容格式
msg.setContent(content, "text/html");
```



单元项目6-创建带附件的邮件

- 项目构思：

- 采用JavaMail API也可以像使用OutLook一样在邮件中添加附件。通过创建包含多个部分的邮件，可以在邮件中带有附件。
- JavaMail API中的BodyPart对象代表着邮件主体的一部分，可以在程序中创建多个BodyPart对象，使用本身的setContent()方法存放附件、视频、音频、图片等。然后使用addBodyPart()方法将该对象加入到MimeMultipart对象中，把创建的每一个BodyPart对象都加入到MimeMultipart对象后，再调用setContent()方法将MimeMultipart对象加入到MimeMessage对象中去。



单元项目6-创建带附件的邮件

- 项目构思：

- BodyPart对象是抽象类，具体使用的是MimeBodyPart对象，如果要向MimeBodyPart对象添加附件需要获得这个附件的数据句柄。此数据句柄通过DataSource对象和具体的附件文件关联。如果附件文件的名称有中文的时候可以用MimeUtility.encodeWord()方法解码，避免附件名称中文乱码。

例：

```
BodyPart attachment=new MimeBodyPart();    //创建MimeBodyPart
String filename="Java技术培训.doc";        //附件文件的名称
//附件的路径
String source=application.getRealPath("")+"\\ "+filename;
//创建数据源指向附件的文件
DataSource ds=new FileDataSource(source);
//获得附件文件的句柄
attachment.setDataHandler(new DataHandler(ds));
//设置附件文件名
attachment.setFileName(MimeUtility.encodeWord(filename));
multipart.addBodyPart(attachment);
```



单元项目6-创建带附件的邮件

- 项目设计
 - 通过一个JSP文件将带有doc文件附件的邮件发送到邮箱中。
- 项目实施
 - 在Web目录下添加“Java技术培训.doc”的文件作为发送邮件的附件。
 - 在项目中通过一个JSP文件实现具体程序。



单元项目6-创建带附件的邮件

• 项目实施

- JSP文件具体程序。

```
Properties props = new Properties();  
props.put("mail.host", "219.216.128.8"); //设置邮件服务器地址  
Session ses = Session.getDefaultInstance(props, null); //创建邮件会话对象  
MimeMessage msg = new MimeMessage(ses); //从Session创建MimeMessage  
InternetAddress from=new InternetAddress("jinyan_t@neusoft.edu.cn");//发送的邮箱地址  
InternetAddress to=new InternetAddress("abc@sohu.com"); //接受的邮箱地址  
msg.setFrom(from); //设置发送邮箱地址  
msg.addRecipient(Message.RecipientType.TO,to); //添加邮箱接受者  
msg.setSubject("带有附件的邮件"); //设置邮件主题  
BodyPart body=new MimeBodyPart(); //创建一个邮件主体内容  
body.setText("带有附件的邮件");  
Multipart multipart=new MimeMultipart();  
multipart.addBodyPart(body); //加入到MimeMultipart中  
BodyPart attachment=new MimeBodyPart(); //创建附件内容  
String filename="Java技术培训.doc"; //附件文件的名字  
String source=application.getRealPath("")+"\\\\"+filename;//附件的路径  
DataSource ds=new FileDataSource(source); //创建数据源指向附件的文件  
attachment.setDataHandler(new DataHandler(ds)); //获得附件文件的指针  
attachment.setFileName(MimeUtility.encodeWord(filename)); //设置附件的文件名  
multipart.addBodyPart(attachment); //加入到MimeMultipart中  
msg.setContent(multipart); //将MimeMultipart加入到Message中  
Transport tran=ses.getTransport("smtp"); //获得传输类的实例  
tran.connect("219.216.128.8", "abc@neusoft.edu.cn", "password");//建立连接  
tran.send(msg); //发送邮件  
out.println("邮件发送成功");
```




单元项目7-在JSP页面中显示接受的邮件

- 项目构思:JavaMail API除了可以发送邮件之外,还提供了很多对其它常见邮件操作的支持,比如接收邮件等。下面是一个具体检索邮件的过程。

- 创建邮件会话:检索邮件需要创建相关的邮件会话。

例: `Properties props = new Properties();`

`props.put("mail.host", "219.216.128.8");`

`Session session = Session.getDefaultInstance(props, null);`

- 创建Store对象:要检索邮件服务器的邮件,需要连接到邮件服务器。这是通过Store对象完成的,通过邮件会话对象可以创建Store对象。Store对象的`connect()`方法用来连接邮件服务器,该方法需要三个参数,第一个参数是邮件服务器的地址,第二个参数是能够登陆邮件服务器的用户名,第三个参数是密码。

例: `Store store= session.getStore("pop3");`

`store.connect("邮件服务器地址","用户名","密码");`

- 文件夹对象Folder:当连接到邮箱后,需要打开存放邮件的文件夹,Folder代表文件夹对象。因此,需要先创建Folder对象。创建Folder对象可以通过调用Store对象的`getFolder()`方法。这个方法有一个参数,代表着要打开文件夹的名字。

例: `Folder folder=store.getFolder("INBOX");`



单元项目7-在JSP页面中显示接受的邮件

- 项目构思：具体检索邮件的过程（续）。
 - 打开文件夹：获得文件夹对象后，要使用open()方法打开文件夹，这个方法有一个参数，可以指定打开文件夹的模式。共有两个可选值：READ_ONLY和READ_WRITE。
例：`folder.open(Folder.READ_ONLY);`
 - 取出文件夹中的邮件：Folder对象的getMessage()或getMessages()提供了在文件夹中收取邮件的方法。
例：`Message[] messages=folder.getMessages();`
 - 读取邮件：通过Message对象提供的方法可以获得整个邮件的信息。例如：`getSubject()`获得邮件的主题、`getSentData()`获得发送的时间、`getContent()`获得具体内容等。
 - 关闭资源：完成接收过程后，需要关闭Folder和Store对象。

例：`folder.close(false);`
`store.close();`



单元项目7-在JSP页面中显示接受的邮件

- 项目设计：将接受到的邮件通过JSP文件显示出来。
- 项目实施：在项目中创建一个JSP页显示接收邮件的内容。



单元项目7-在JSP页面中显示接受的邮件

- 项目实施：接收邮件的内容的代码。

```
Properties props = new Properties();
props.put("mail.host", "219.216.128.8");    //设置邮件服务器地址
//创建邮件会话对象
Session ses = Session.getDefaultInstance(props, null);
Store store=ses.getStore("pop3");
store.connect("219.216.128.8", "abc@neusoft.edu.cn", "密码");
Folder folder=store.getFolder("INBOX");    //创建文件夹对象
folder.open(Folder.READ_ONLY); //打开文件夹
Message[] messages=folder.getMessages();
for(int i=0;i<messages.length;i++){    //读取文件夹中所有的邮件信息
    Message msg=messages[i];
    out.println("主题: "+msg.getSubject()+"<br>");
    out.println("发送时间: "+msg.getSentDate()+"<br>");
    out.println("内容: "+msg.getContent());
    out.println("<hr>");
}
folder.close(false);
store.close();
```



JavaMail

- 邮件的删除：删除邮件服务器上的邮件步骤：
 - 以READ_WRITE模式打开文件夹：要在收取之后删除邮件，必须使用READ_WRITE模式打开文件夹。可以用下面的代码代替上个程序中打开文件夹的代码。
- 例：`folder.open(Folder.READ_WRITE);`
- 设置邮件状态：删除邮件必须用系统标记标识邮件的状态。标记描述了邮件在其所在文件夹中的状态。`Message.getFlags()`方法返回一个Flag对象，该对象拥有该邮件所有当前设置的标记。
- 完成删除操作：要完成删除操作，还必须给关闭文件夹的方法一个true值的参数，这样在文件夹关闭时所有标记是DELETED的邮件将被删除。修改上个程序的代码。

例：`folder.close(true);`

第13章 Java Web开发常用功能

- 文件上传
- 分页处理
- JavaMail
- 树状菜单



树状菜单

- 树状菜单常用来提供内容导航功能。
- 它的实现分为静态和动态两个部分。
 - 静态菜单是内容固定不变或基本不变，可以采用JavaScript实现，这种方法的特点是速度快，编程难度小，有许多开发好的程序。
 - 动态树状菜单是树状菜单的内容经常变化，需要动态生成。

单元项目8-采用菜单组件创建静态树状菜单

- 项目构思：

- 静态树状菜单可以采用JavaScript开发，网上有许多优秀的代码程序，这里介绍一个比较简单实用的免费树状菜单dtree组件。
- dtree组件的主要优点是：可以设置无限级的菜单、可以设置多种状态(如：图标的显示和隐藏等)、支持目前主流的浏览器、节点图片可以设置切换图片的效果等。下载地址是：
<http://www.destroydrop.com/javascripts/tree/>。
- dtree组件下载的文件包括dtree.js、dtree.css和图片文件夹img。



单元项目8-采用菜单组件创建静态树状菜单

- 项目设计：在一个JSP页面中通过dtree组件创建一个树状菜单。
- 项目实施：
 - 将下载的dtree组件dtree.js、dtree.css、img中的图片文件分别放到相应的WEB目录下的js、css、img文件夹中。
 - 在JSP页面中实现代码。



单元项目8-采用菜单组件创建静态树状菜单

- 项目实施：
 - 在JSP页面中实现的主要代码如下：

```
<script type="text/javascript">
    d = new dTree('d');
    d.add(0,-1,'我的树状菜单');
    d.add(1,0,'节点1','tree.jsp');
    d.add(2,0,'节点2','tree.jsp');
    d.add(3,1,'节点1.1','tree.jsp');
    d.add(4,0,'节点3','tree.jsp');
    d.add(5,3,'节点1.1.1','tree.jsp');
    d.add(6,5,'节点1.1.1.1','tree.jsp');
    d.add(7,0,'节点4','tree.jsp');
    d.add(8,1,'节点1.2','tree.jsp');
    d.add(9,0,'我的相册','tree.jsp','这是我的相册
                                     ',' ',' ','img/imgfolder.gif');
    d.add(10,9,'我的生日','tree.jsp','我的生日照片');
    d.add(11,9,'北京旅游','tree.jsp');
    d.add(12,0,'回收站','tree.jsp',' ',' ','img/trash.gif');
    document.write(d);
</script>
```



单元项目9-采用菜单组件创建动态树状菜单

- 项目构思：

- 动态树状菜单的内容是通过程序生成，需要在JSP或Servlet中生成JavaScript代码，在上面程序的基础上采用动态方法生成树状菜单。

- 项目设计：

- 根据dtree组件生成节点的参数，需要在数据库中创建一个树状结构表，作为树状菜单节点的参数。
- 创建一个JavaBean，在JavaBean的方法中读取表中的数据，生成相应的JavaScript代码创建出树状菜单。
- 在JSP页面中利用JSP动态标签调用JavaBean，在页面显示生成的树状菜单。



单元项目9-采用菜单组件创建动态树状菜单

- 项目实施：

- 创建树状结构表DTREE。其字段为生成树状节点中的8个参数。

```
create table dtree (  
    id int(11) not null default 0,  
    pid int(11) not null default -1,  
    name varchar(50) not null,  
    url varchar(50) default '',  
    title varchar(50) default '',  
    target varchar(50) default '_self',  
    icon varchar(50) default null,  
    iconopen varchar(50) default null,  
    primary key (id)  
) ENGINE=InnoDB DEFAULT CHARSET=gbk
```



单元项目9-采用菜单组件创建动态树状菜单

- 项目实施：

- 在创建的表中插入相应的数据，这些将作为生成树状菜单节点参数，在程序中通过读取表中的数据生成树状菜单的节点。
- 创建一个JavaBean， 通过JavaBean读取数据库中的数据，根据表中的数据生成相应树状菜单的JavaScript代码，返回到页面。



单元项目9-采用菜单组件创建动态树状菜单

- 项目实施:

- JavaBean中getTreeNodes()的主要代码:

```
StringBuffer buf = new StringBuffer();           //保存生成的JavaScript代码
buf.append("<script type='text/javascript'>"); //生成JavaScript标记
buf.append("d = new dTree('d');");              //创建一个dtree
Connection conn = getConnection();               //建立连接对象
try {
    Statement stmt = conn.createStatement();
    String sql = "select * from dtree";
    ResultSet rs = stmt.executeQuery(sql);
    while (rs.next()) {                           //根据结果集的内容创建一个节点
        buf.append("d.add(");
        buf.append(m.get("id") + ",");
        buf.append(m.get("pid") + ",");
        buf.append(m.get("name") + "', '");
        buf.append(m.get("url") + "', '");
        buf.append(m.get("title") + "', '");
        buf.append(m.get("target") + "', '");
        buf.append("img/" + m.get("icon") + "', '");
        buf.append("img/" + m.get("iconopen"));
        buf.append("');");
    }
} catch (SQLException e) {e.printStackTrace();}
buf.append("document.write(d);");
buf.append("</script>");
.....省略部分代码
```


单元项目9-采用菜单组件创建动态树状菜单

- 项目实施：
 - 在JSP页面中调用JavaBean生成树状菜单。

```
<html>
<head>
  <title>dtree sample</title>
  <!-- 导入dtree组件的样式表dtree.css和js文件dtree.jsp -->
  <link rel="StyleSheet" href="css/dtree.css" type="text/css" />
  <script type="text/javascript" src="js/dtree.js"></script>
</head>
<body>
  <div class="dtree" id="divTree">
    <br><a href="javascript: d.openAll();">全部展开</a> |
    <a href="javascript: d.closeAll();">全部折叠</a><br/>
    <jsp:useBean id="tree" class="com.dtree.util.TreeUtil">
      <jsp:getProperty name="tree" property="treeNodes" />
    </jsp:useBean>
  </div>
</body>
</html>
```



小结

- 本章主要介绍了Java Web开发中常用的技术。
 - 文件的上传
 - 分页技术
 - 使用JavaMail API收发邮件
 - 树状菜单的使用
- 通过案例代码详细介绍了这些技术中主要的功能。